МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

к лабораторной работе по дисциплине «Языки программирования»

«Создание языка программирования.
Лексический анализ программного кода»

1. Цель и задачи работы

Цель работы – изучение принципов построения лексических анализаторов и приобретение практических навыков разработки компонентов компилятора на языке программирования Python.

В ходе выполнения работы студент должен решить следующие задачи:

1. спроектировать словарь разрабатываемого мини-языка: составить перечень ключевых слов, категорий литералов, операторов и разделителей;
2. формализовать каждую категорию лексем в виде регулярного выражения;
3. реализовать лексический анализатор, формирующий таблицу токенов с координатами каждой лексемы (строка, столбец);
4. реализовать диагностику лексических ошибок с указанием позиции и описанием причины;
5. подготовить комплект позитивных и негативных тестовых примеров и провести тестирование;
6. оформить отчёт и подготовиться к защите работы.

2. Теоретическая часть

2.1. Место лексического анализа в обработке программы

Лексический анализ – первый этап обработки текста программы. На входе лексер получает последовательность символов исходного кода, на выходе формирует поток токенов – единиц, с которыми уже работает синтаксический анализатор. Общая схема обработки программы приведена на рисунке 1.

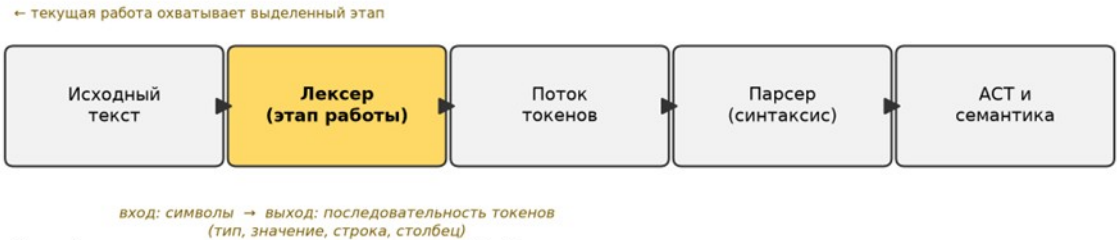


Рисунок 1 – Схема обработки программы и место лексического анализа в ней

Выделение лексики в отдельный этап упрощает каждый из компонентов: синтаксический анализатор оперирует укрупнёнными единицами и не тратит время на пропуски пробелов, разбор комментариев и слияние символов в лексемы; сообщения об ошибках формируются с точностью до строки и столбца. Важно понимать, что лексер и синтаксический анализатор отвечают за разные уровни корректности: так, несоответствие числа открывающих и закрывающих скобок обнаружит именно парсер, а не лексер – лексер лишь добросовестно выдаст каждую скобку отдельным токеном.

2.2. Основные понятия: лексема, токен, шаблон, словарь

Лексема – минимальная значимая последовательность символов текста программы (например, сумма, 3.14, +=). Токен – внутреннее представление лексемы, содержащее, как правило, тип, значение и позицию в исходном файле. Шаблон (pattern) – правило, по которому распознаются лексемы данной категории. Словарь языка – совокупность всех шаблонов и ключевых слов. Взаимосвязь понятий показана на рисунке 2.

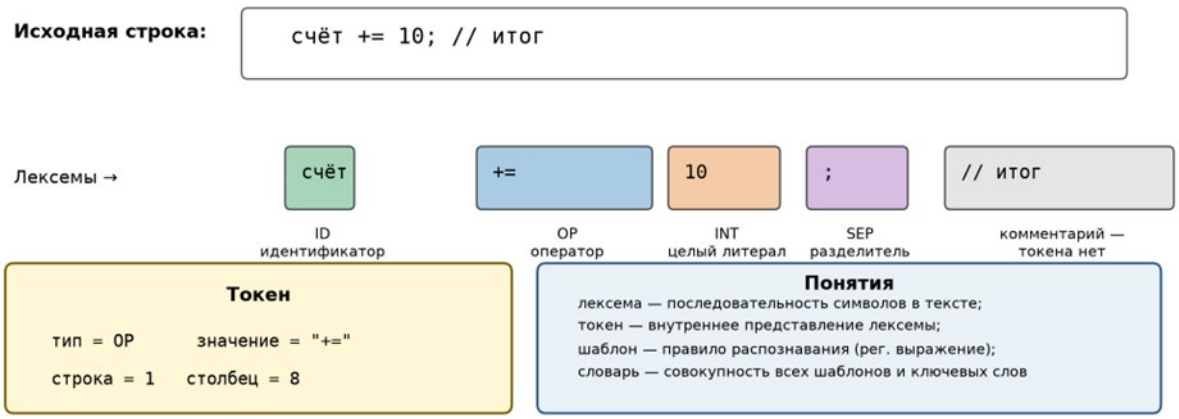


Рисунок 2 – Основные понятия лексического анализа на примере строки «сумма += 10; // итог»

Пример: во фрагменте сумма += 10 лексема += порождает токен типа OP со значением +=, лексема 10 – токен типа INT со значением 10, а пробелы не порождают токенов вовсе – они лишь разделяют лексемы.

2.3. Как лексер узнаёт лексемы: посимвольное чтение, просмотр вперёд, максимальное совпадение

Лексер читает исходный текст символ за символом, храня текущую позицию (номер строки и столбца). На каждом шаге он смотрит на текущий символ и решает, к какой категории тот может относиться:

- буква или подчёркивание – возможно, начинается идентификатор (или ключевое слово);
- цифра – возможно, начинается числовой литерал;
- кавычка – начинается строковый литерал;
- косая черта – это оператор деления или начало комментария (решает просмотр следующего символа);
- иной символ – либо разделитель, либо (при отсутствии другого варианта) ошибка.

Узнав категорию, лексер накапливает символы, пока они «продолжают» лексему: у идентификатора это буквы, цифры и подчёркивание, у числа – цифры, точка и экспонента. Первый неподходящий символ завершает лексему и остаётся на входе для следующего шага.

Регулярные выражения в этой схеме – не отдельный механизм, а удобная запись того самого правила «из чего состоит лексема». Синтаксис, которого достаточно для словаря учебного языка:

Конструкция	Смысл
[abc]	любой один символ из перечисленных: a, b или c
[^abc]	любой символ, кроме перечисленных
[0-9]	диапазон: любая цифра
*	предыдущая конструкция повторяется 0 и более раз
+	предыдущая конструкция повторяется 1 и более раз
?	предыдущая конструкция встречается 0 или 1 раз
|	выбор одного из вариантов («или»)

Регулярные выражения базового языка «Бауманец» с переводом на человеческий язык:

Категория	Регулярное выражение	Правило словами
Идентификатор	[A-Za-z_][A-Za-z0-9_]*	сначала буква или «_», затем любое количество букв, цифр и «_»
Целое десятичное	0 | [1-9][0-9]*	ноль или ненулевая цифра, за которой идут цифры
Целое 16-ричное	0[xX][0-9a-fA-F]+	префикс 0x или 0X, затем хотя бы одна шестнадцатеричная цифра
Целое двоичное	0[bB][01]+	префикс 0b или 0B, затем хотя бы один бит
Вещественное	[0-9]+\.[0-9]+([eE][+-]?[0-9]+)? | [0-9]+[eE][+-]?[0-9]+	цифры, точка, цифры, необязательная экспонента; либо целое с экспонентой
Строка	"([^"\\\n]|\\.)*"	кавычка, затем любые символы кроме кавычки и перевода строки (экранированные – можно), закрывающая кавычка
Комментарий //	//[^\n]*	два слэша и всё до конца строки
Комментарий /*…*/	/*([^*]|*[^/])**/	открывающая пара, всё, что не содержит «*/», закрывающая пара

Трассировка: как лексер читает строку «сумма += 10;»

Рассмотрим по шагам, как описанная схема работает на конкретном входе. Пусть лексер читает строку сумма += 10; (после точки с запятой – конец файла). Пошаговая трассировка приведена в таблице.

Шаг	Символ	Что видит лексер	Действие	Результат
1	с	буква – возможное начало идентификатора	начинает накопление лексемы	идёт чтение

Шаг	Символ	Что видит лексер	Действие	Результат
2	у, м, м, а	буквы – продолжают идентификатор	продолжает накопление	лексема «сумма» растёт
3	(пробел)	не буква, не цифра и не «_»	завершает лексему; проверяет по списку ключевых слов – совпадения нет	токен ID(сумма)
4	(пробел)	пробельный символ	пропускает, не порождая токена	–
5	+	оператор; впереди – символ «=»	проверяет пару «+=» в списке двухсимвольных операторов; пара найдена	читает второй символ, токен OP(+=)
6	(пробел)	пробельный символ	пропускает	–
7	1	цифра – возможное начало числа	начинает накопление числа	идёт чтение
8	0	цифра – продолжает число	продолжает накопление	лексема «10» растёт
9	;	не цифра – число закончилось	завершает лексему	токен INT(10)
10	;	разделитель	сохраняет как есть	токен SEP(;)

Трассировка наглядно объясняет три ключевых правила. Во-первых, границы лексем находятся не заранее, а постфактум: лексер узнаёт, что идентификатор закончился, лишь встретив первый «чужой» символ. Во-вторых, двухсимвольные операторы требуют просмотра на один символ вперёд: без него шаг 5 превратил бы «+=» в два оператора «+» и «=». В-третьих, пробелы не порождают токенов, но именно они делают возможным завершение лексем: во входе сумм а без пробела лексер соберёт единый идентификатор «сумма».

Для закрепления полезно самостоятельно протрассировать вход 123abc. Лексер накапливает цифры 123, а затем встречает букву – для числового литерала буква «чужой» символ. Ошибки при этом нет: число завершается, выдаётся токен INT(123), а с буквы начинается новый идентификатор abc. Иная ситуация на входе 0x без цифр: после префикса не найдено ни одной шестнадцатеричной цифры, накопить литерал невозможно – фиксируется ошибка «ожидаются цифры после 0x».

2.4. Подходы к реализации лексера

Подход	Плюсы	Минусы
Ручной посимвольный разбор	Полный контроль над поведением; нагляден; легко вести статистику позиций	Много кода; легко допустить ошибку в границах лексем

Подход	Плюсы	Минусы
Разбор по регулярным выражениям (модуль re)	Краткость; шаблоны совпадают с описанием словаря; простая отладка	Трудно выразить контекст (вложенные комментарии); ниже скорость на больших файлах
Генераторы анализаторов (PLY, SLY, ANTLR)	Автоматическая генерация из описания; промышленное качество	Порог вхождения выше; поведение «внутри генератора» менее прозрачно

В базовом варианте работы используется ручной посимвольный разбор: он позволяет в явном виде пронаблюдать все тонкости (просмотр вперёд, границы лексем, подсчёт координат). Сравнение с вариантом на модуле re выносится в индивидуальное задание.

2.5. Таблица токенов и диагностика ошибок

Результат работы лексера – таблица токенов: упорядоченный список записей вида «тип, значение, строка, столбец». Комментарии и пробельные символы в таблицу не включаются, однако они влияют на подсчёт координат.

Типовые лексические ошибки: недопустимый символ; незакрытый строковый литерал (в том числе достижение конца файла); перевод строки внутри строкового литерала; незакрытый блочный комментарий; некорректный числовой литерал (0x без цифр, экспонента без цифр, «висящая» точка); цифра в начале идентификатора.

2.6. Рабочая среда: OpenIDE на Astra Linux

Работа выполняется в среде OpenIDE под операционной системой Astra Linux. Проверьте доступность интерпретатора: откройте встроенный терминал среды и выполните команду python3 --version; убедитесь, что установлена версия 3.10 или выше.

- создайте в рабочей области проекта каталог lab_lex с файлами tokens.py, lexer.py, main.py и каталогом tests (структура приведена в разделе 4.1);
- все файлы сохраняйте в кодировке UTF-8: словарь языка содержит кириллические ключевые слова (если, пока, программа и другие), и неверная кодировка сделает их нераспознаваемыми;
- запуск программы выполняйте из встроенного терминала среды командой python3 main.py tests/positive_01.src – так же, как и отладочный вывод, он виден в панели вывода OpenIDE;
- тестовые файлы размещайте в каталоге tests с расширением .src, чтобы редактор не подсвечивал их как Python-код.

Перед сдачей работы убедитесь, что проект открывается и запускается на чистой копии: проверяющий воспроизводит запуск по инструкции из отчёта.

3. Постановка задания

В работе разрабатывается лексический анализатор для учебного мини-языка «Бауманец». Язык – С-подобный: без классов и указателей, с обязательной точкой с запятой и блоками в фигурных скобках.

Состав словаря базового языка:

- ключевые слова (8): если, иначе, пока, возврат, функция, истина, ложь, программа; терминатор программы – слово «конец»;
- идентификаторы: латинские буквы, цифры, символ «_»; начинаются с буквы или «_»; регистрозависимые;
- целые литералы: десятичные, а также с префиксами 0x (шестнадцатеричные) и 0b (двоичные);
- вещественные литералы: обычная форма (3.14) и экспоненциальная (1e-5, 2.5E+3);
- строковые литералы: в двойных кавычках, с escape-последовательностями \n, \t, \" , \\ ;
- операторы: арифметические + - * / % ; сравнения == != < > <= >= ; присваивание = ; составное присваивание += -= ; логические && || ! ; инкремент ++ ;
- разделители: ( ) { } [ ] , ; .
- комментарии: однострочные // и блочные /* */; блочные комментарии не вкладываются.

Минимальный состав разрабатываемого лексера – не менее 10 различных типов токенов из перечисленных категорий.

Пример программы на языке «Бауманец»

```
программа
    счёт : целое = 0;
    пока (счёт < 10) {
        если (счёт % 2 == 0) {
            печать("чётное\n");
        } иначе {
            счёт += 1;
        }
    }
конец
```

Фрагмент таблицы токенов для второй строки примера

№	Лексема	Тип токена	Строка	Столбец
1	счёт	идентификатор	2	5
2	:	разделитель	2	9
3	целое	идентификатор (тип)	2	11
4	=	оператор	2	17
5	0	целый литерал	2	19
6	;	разделитель	2	20

Требования к программе: вход – текстовый файл с исходным кодом; выход – таблица токенов (на экран или в файл); при обнаружении лексической ошибки выводится сообщение с номером строки, столбца и описанием. Обработка ошибки по согласованию с преподавателем: с остановкой или с продолжением разбора последующего текста (рекомендуется второе – так за один прогон выявляются все ошибки файла).

4. Пошаговая реализация на Python

4.1. Структура проекта

```
lab_lex/
├── tokens.py     - класс Token, константы словаря
├── lexer.py      - класс Lexer, распознавание лексем
├── main.py       - чтение файла, вывод таблицы токенов
└── tests/        - позитивные и негативные примеры (.src-файлы)
```

4.2. Класс Token

Токен удобно оформить классом-обёрткой: это даёт единый формат печати таблицы и сравнения в тестах.

```
from dataclasses import dataclass

@dataclass
class Token:
    type: str      # KW, ID, INT, FLOAT, STR, OP, SEP, EOF
    value: str     # исходный текст лексемы
    line: int      # номер строки (с 1)
    col: int       # номер столбца (с 1)

    def __repr__(self) -> str:
        return f"{self.type}({self.value!r}) [{self.line}:{self.col}]"
```

4.3. Каркас лексера

Центральные методы класса Lexer: advance() продвигает позицию и ведёт учёт строк и столбцов; peek(offset) возвращает символ на расстоянии offset без продвижения (просмотр вперёд). Главный цикл разбирает вход по первому символу:

```
KEYWORDS = {"если", "иначе", "пока", "возврат", "функция",
            "истина", "ложь", "программа"}
SINGLE = set("+-*/%=<>!(){}[],;.")
DOUBLE = {"==", "!=", "<=", ">=", "+=", "-=", "&&", "||", "++"}

class Lexer:
    def __init__(self, source: str):
        self.src = source
        self.pos = 0        # индекс текущего символа
        self.line = 1
        self.col = 1

    def advance(self) -> str:
        ch = self.src[self.pos]
        self.pos += 1
        if ch == "\n":
            self.line += 1
            self.col = 1
```

```
        else:
            self.col += 1
        return ch

    def peek(self, offset: int = 0) -> str:
        i = self.pos + offset
        return self.src[i] if i < len(self.src) else ""

    def tokenize(self) -> list[Token]:
        tokens = []
        while self.pos < len(self.src):
            ch = self.peek()
            if ch.isspace():
                self.advance()
            elif ch == "/" and self.peek(1) == "/":
                self.skip_line_comment()
            elif ch == "/" and self.peek(1) == "*":
                self.skip_block_comment()
            elif ch.isdigit() or (ch == "." and self.peek(1).isdigit()):
                tokens.append(self.read_number())
            elif ch.isalpha() or ch == "_":
                tokens.append(self.read_identifier())
            elif ch == '"':
                tokens.append(self.read_string())
            elif ch in SINGLE:
                tokens.append(self.read_operator())
            else:
                raise LexError(f"недопустимый символ {ch!r}",
                               self.line, self.col)
        tokens.append(Token("EOF", "", self.line, self.col))
        return tokens
```

Обратите внимание: проверка комментария стоит раньше проверки оператора деления – иначе символ / всегда будет порождать токен OP.

Как проверить себя

После каркаса лексер должен корректно разбирать строку счёт : целое = 0; на шесть токенов. Если таблица совпала с эталоном из раздела 3 – двигайтесь дальше. Промежуточную самопроверку удобно вести трассировкой из раздела 2.3: прогоните вручную строку сумма += 10; и сравните свой пошаговый разбор с таблицей.

4.4. Операторы: просмотр вперёд и максимальное совпадение

Символы «=», «<», «+» могут начинать как однобуквенные, так и двухбуквенные операторы. Решение – сначала проверить двухсимвольный вариант:

```
def read_operator(self) -> Token:
    line, col = self.line, self.col
    one = self.advance()
    two = one + self.peek()
    if two in DOUBLE:              # сначала пробуем двухсимвольный оператор
        self.advance()
        return Token("OP", two, line, col)
    return Token("OP", one, line, col)
```

Типичная ошибка – обратный порядок проверок: тогда вход a<=b разобьётся на токены < и =. Вход a==b, разобранный как два оператора =, – вторая классическая ловушка.

4.5. Числовые литералы

Разбор числа реализует правило из раздела 2.3. Ловушки: вход 123abc не должен порождать ошибку – после цифр идёт идентификатор; 0x без последующих цифр – ошибка; экспонента без цифр – ошибка.

```
def read_number(self) -> Token:
    line, col, start = self.line, self.col, self.pos
    if self.peek() == "0" and self.peek(1) in "xXbB":
        prefix = self.advance() + self.advance()
        digits = "0123456789abcdefABCDEF" \
            if prefix.lower() == "0x" else "01"
        if self.peek() not in digits:
            raise LexError(f"ожидаются цифры после {prefix!r}", line, col)
        while self.peek() in digits:
            self.advance()
    else:
        while self.peek().isdigit():
            self.advance()
        if self.peek() == "." and self.peek(1).isdigit():
            self.advance()
            while self.peek().isdigit():
                self.advance()
        if self.peek() in "eE":
            self.advance()
            if self.peek() in "+-":
                self.advance()
            if not self.peek().isdigit():
                raise LexError("ожидаются цифры в экспоненте",
                               self.line, self.col)
            while self.peek().isdigit():
                self.advance()
    lexeme = self.src[start:self.pos]
    kind = "FLOAT" if any(c in ".eE" for c in lexeme) else "INT"
    return Token(kind, lexeme, line, col)
```

4.6. Идентификаторы и ключевые слова

```
def read_identifier(self) -> Token:
    line, col, start = self.line, self.col, self.pos
    while self.peek().isalnum() or self.peek() == "_":
        self.advance()
    lexeme = self.src[start:self.pos]
    kind = "KW" if lexeme in KEYWORDS else "ID"
    return Token(kind, lexeme, line, col)
```

Проверку на ключевое слово удобно выполнять после сбора всей лексемы: словарь ключевых слов хранится в множестве, поэтому поиск выполняется за константное время.

4.7. Строковые литералы

```
def read_string(self) -> Token:
    line, col = self.line, self.col
    self.advance()                       # открывающая кавычка
    parts = []
    escapes = {"n": "\n", "t": "\t", '"': '"', "\\": "\\"}
    while True:
        ch = self.peek()
        if ch == "":
            raise LexError("строка не закрыта до конца файла", line, col)
```

```
        if ch == "\n":
            raise LexError("перевод строки внутри литерала",
                           self.line, self.col)
        if ch == '"':
            self.advance()
            return Token("STR", "".join(parts), line, col)
        if ch == "\\":
            self.advance()
            esc = self.peek()
            if esc not in escapes:
                raise LexError(f"неизвестная escape-последовательность \\{esc}",
                               self.line, self.col)
            parts.append(escapes[esc])
            self.advance()
        else:
            parts.append(self.advance())
```

В таблице токенов значение строкового литерала храните уже после обработки escape-последовательностей, а в тестах сверяйте именно его.

4.8. Комментарии

```
def skip_line_comment(self):
    while self.peek() not in ("", "\n"):
        self.advance()

def skip_block_comment(self):
    line, col = self.line, self.col
    self.advance(); self.advance()       # символы /*
    while self.peek() != "":
        if self.peek() == "*" and self.peek(1) == "/":
            self.advance(); self.advance()
            return
        self.advance()
    raise LexError("незакрытый блочный комментарий", line, col)
```

Проверьте поведение на входе /* вложенный /* комментарий */ всё ещё комментарий */: в базовом языке вложенность запрещена, поэтому комментарий закрывается первым же */ , а последующий текст разбирается как обычный код.

4.9. Подсчёт координат и тестирование

Координаты считаются в advance(): при встрече \n увеличивается номер строки и столбец сбрасывается в 1. Символ табуляции по согласованию засчитывается за 4 позиции (или 8) – зафиксируйте решение в отчёте.

Тестирование ведите по двум комплектам файлов: позитивные примеры (корректные программы, для каждой – эталонная таблица токенов) и негативные (входы с ошибками из раздела 2.5, для каждого – ожидаемый текст ошибки и её координаты). Автоматизируйте проверку средствами pytest: каждый негативный пример – отдельный тест.

5. Варианты индивидуальных заданий

Каждый вариант представляет собой модификацию базового языка «Бауманец», описанную в разделе 3. Ниже для каждого варианта изложены: суть изменения словаря, пояснения к реализации – какие методы каркаса из раздела 4 затрагиваются и на какие тонкости следует

обратить внимание – и обязательный комплект негативных тестов. К своему варианту студент прилагает не менее трёх негативных тестовых примеров с разными типами лексических ошибок. Во всех вариантах общие методы advance(), peek() и подсчёт координат из раздела 4.9 не меняются.

Вариант 1. Вложенные блочные комментарии

Суть изменения. Блочные комментарии разрешается вкладывать: вход /* a /* b */ c */ считается одним комментарием целиком, а его внутренность не порождает токенов.

Реализация. Изменению подвергается только метод skip_block_comment(). Вместо поиска первой пары «*/» вводится счётчик глубины: каждая встреченная пара «/*» увеличивает его на единицу, «*/» – уменьшает; комментарий завершён, когда счётчик обнулился. Оба срабатывания проверяются через peek(0) и peek(1), как и в базовой версии. Важно: на входе без вложенности поведение должно совпадать с базовым вариантом – это первый тест.

Обязательные негативные тесты: незакрытый комментарий (конец файла при ненулевом счётчике); лишняя закрывающая пара «*/» вне комментария – обсудите в отчёте, во что она превращается (оператор «*» и оператор «/»); комментарий с одиночной звёздочкой внутри «/* * /» – пара «*/» разорвана пробелом и закрытия не происходит.

Вариант 2. Символьные литералы в одинарных кавычках

Суть изменения. В словарь добавляется категория CHAR: литерал вида 'a', '\n', содержащий ровно один символ или escape-последовательность.

Реализация. В главном цикл tokenize() добавляется ветка elif ch == " ' ": вызывающая новый метод read_char(). Метод читает открывающую кавычку, затем ровно один элемент: либо escape-пару через таблицу escapes из раздела 4.7, либо одиночный символ, – и требует закрывающую кавычку. Обработку escape-последовательностей целесообразно выделить в общий вспомогательный метод, чтобы не дублировать код со read_string().

Обязательные негативные тесты: пустой литерал '' (между кавычками нет символа); литерал 'ab' с двумя символами; строка 'a без закрывающей кавычки – до конца файла.

Вариант 3. Числа с разделителями разрядов 1_000_000

Суть изменения. Внутри числового литерала разрешаются разделители разрядов: 1_000_000, 0xFF_FF, 3.14_15. Разделитель не может стоять в начале, в конце числа и подряд.

Реализация. Изменению подвергается read_number(). После каждого прочитанного блока цифр проверяется: если peek() == "_" и peek(1) – цифра данного набора (для шестнадцатеричных – своя таблица digits), разделитель и следующая цифра читаются. Ключевой момент: символ «_» остаётся допустимым началом идентификатора, поэтому вход _100 не должен попадать в ветку чисел – проверьте порядок ветвей в tokenize().

Обязательные негативные тесты: 1_ – разделитель в конце, за ним нет цифры; 0x_1F – разделитель сразу после префикса; 1__0 – два разделителя подряд.

Вариант 4. Кириллические идентификаторы

Суть изменения. Идентификаторы могут содержать буквы кириллицы наравне с латиницей: счётчик_1, Температура. Ключевые слова при этом остаются регистрозависимыми.

Реализация. Стандартные методы isalpha() и isalnum() в Python принимают и кириллицу, поэтому формально каркас «заработает» без изменений – но студен должен показать, что контролирует набор символов сознательно. Замените проверки на собственный предикат is_ident_char(ch), явно перечисляющий диапазоны: латиница, А-Я, а-я, Ё-ё, цифры, подчёркивание. Это снимает вопрос «а что ещё пропустит isalnum?» – например, греческие буквы или цифры других алфавитов.

Обязательные негативные тесты: идентификатор «Если» – совпадает с ключевым словом «если» с точностью до регистра и потому является идентификатором (проверьте, что лексер не спутает); цифра в начале «2путь»; слово с дефисом «счёт-чик» – распадается на два идентификатора и оператор «-».

Вариант 5. Оператор степени ** и диапазон ..

Суть изменения. В таблицу операторов добавляются двухсимвольные ** и .., используемые как оператор возведения в степень и оператор диапазона.

Реализация. Расширяется множество DOUBLE, но этого мало: появляется конфликт между точкой-разделителем и началом диапазона. В ветке чисел уже есть проверка (ch == "." and self.peek(1).isdigit()) – она защищает от «висящей» точки, однако вход 1..5 требует особого порядка: read_number прочитает «1» и остановится, поскольку после точки стоит не цифра; затем read_operator увидит точку и по peek(1) = "." вернёт оператор «..». Проверьте трассировкой, что «1..=5» в варианте 12 не ломает этот порядок.

Обязательные негативные тесты: *** – оператор «**» плюс одиночный «*»; одинокая точка в конце строки после идентификатора – по правилам языка это ошибка (уточните у преподавателя, если точка разрешена как завершитель); вход 1.5. – две точки в одном числе.

Вариант 6. Heredoc-строки в тройных кавычках

Суть изменения. Наряду с обычными строками допускаются многострочные литералы в тройных кавычках """ … """, в которых разрешены переводы строк и любые кавычки.

Реализация. В ветке строк главного цикла сначала проверяется peek(1) и peek(2): если три кавычки подряд – вызывается read_triple_string(), иначе обычный read_string(). Многострочность означает, что внутри литерала advance() продолжает услежать координаты, а проверка на «\n» из read_string() здесь не выполняется. Конец литерала – первая встреченная тройная кавычка; вложенные одинарные и двойные кавычки внутри просто копируются в значение.

Обязательные негативные тесты: незакрытая тройная кавычка до конца файла; литерал """abc"" – две закрывающие кавычки из трёх, за ними обычный текст; пустая тройная строка, совпадающая с открывающей последовательностью another""""" – обсудите в отчёте, сколько кавычек съедается при закрытии.

Вариант 7. Регистронезависимые ключевые слова

Суть изменения. Ключевые слова распознаются без учёта регистра: Если, ПОКА, воЗврат – те же ключевые слова; идентификаторы при этом остаются регистрозависимыми: счёт и Счёт – разные переменные.

Реализация. Изменяется одна строка read_identifier(): перед поиском в KEYWORDS лексема приводится к нижнему регистру методом lower(). Тонкость в том, что нормализовать нужно только проверку на ключевое слово, но не хранимое значение токена: в таблице токенов лексема

сохраняется как написано в исходнике. Для кириллицы метод lower() корректен в Python «из коробки» – убедитесь в этом тестом на слове «ИНАЧЕ».

Обязательные негативные тесты: идентификатор «Если» – теперь это ключевое слово, и программа с переменной Если должна вызвать ошибку уже на лексическом уровне (согласуйте с преподавателем, где её ловить); слово «ПоКа» как имя переменной; убедитесь, что вход «ПОКА» дал тип KW, а не ID.

Вариант 8. Документирующие комментарии / */ и аннотации @имя**

Суть изменения. Добавляются два элемента словаря: документирующий комментарий «/** … */» (порождает отдельный токен DOC) и аннотации @имя (токен ANNOT), допустимые только перед объявлениями.

Реализация. В ветке комментария проверяется peek(2): если после «/*» следует «*», вызывается метод чтения документирующего комментария, порождающий токен DOC вместо тихого пропуска. Для аннотаций в tokenize() добавляется ветка «@»: за символом обязан следовать идентификатор, иначе ошибка; сам префикс читается вместе с именем в один токен. Проверку «допустима ли аннотация в этой позиции» лексер не делает – объясните в отчёте, почему это задача синтаксического анализатора.

Обязательные негативные тесты: символ «@» в начале выражения, за которым нет идентификатора; незакрытый документирующий комментарий; вход «@=x» – аннотация не может начинаться со знака равенства.

Вариант 9. Суффиксы литералов 42L, 3.5f

Суть изменения. Числовые литералы могут заканчиваться суффиксом типа: L – длинное целое, f – вещественное. Суффикс является частью литерала, а не началом идентификатора.

Реализация. В конце read_number(), перед формированием токена, проверяется peek(): если это «L» или «f» (набор суффиксов согласуется с преподавателем), символ читается и включается в лексему, а тип токена уточняется суффиксом. Тонкость: конфликт с идентификаторами – вход x = 5i не должен порождать идентификатор «i», если суффикс «i» не разрешён; заодно обсудите, что произойдёт с входом 42Layer – по принципу максимального совпадения суффикс «L» нельзя отрезать, если далее идут буквы: правильное решение – воспринять «42» как число и «Layer» как идентификатор.

Обязательные негативные тесты: 42LL – двойной суффикс; 3.5g – неизвестный суффикс; суффикс, приставший к идентификатору, в который нельзя превратиться по правилам варианта.

Вариант 10. Операторы сдвига << >> и >>>

Суть изменения. Добавляются операторы побитового сдвига << и >>, а также беззнаковый сдвиг >>> (три одинаковых символа).

Реализация. В read_operator() перед проверкой двухсимвольного варианта добавляется проверка трёхсимвольного: если peek(0), peek(1) и peek(2) – одинаковые символы сдвига, читаются три символа. Порядок проверок: три, затем два, затем один – это прямо иллюстрирует принцип максимального совпадения на последовательности одинаковых символов.

Обязательные негативные тесты: <<< – оператор «<<» плюс одиночный «<»; вход >>< – оператор «>>» плюс «<»; последовательность «>=» после «>» – проверьте, что она не съедается как сдвиг.

Вариант 11. Директивы препроцессора в начале строки

Суть изменения. Строки вида # include, # define распознаются как директивы: директива обязана начинаться символом «#» в начале строки (допустимы пробелы перед ним) и занимает всю строку до перевода строки.

Реализация. В tokenize() перед общим разбором отслеживается признак «начала строки»: если col == 1 или с начала строки прочитаны только пробелы, а peek() == "#", вызывается read_directive(): весь остаток строки поглощается в один токен DIR, имя директивы проверяется по множеству разрешённых (include, define). Ключевой момент – контекст начала строки не выражается одиночным символом, поэтому лексеру нужен флаг состояния, сбрасываемый после каждого «\n».

Обязательные негативные тесты: символ «#» в середине строки – ошибка «неожиданный символ»; неизвестная директива # define2; символ «#» без имени директивы.

Вариант 12. Оператор => и диапазон ..=

Суть изменения. Добавляются трёхсимвольная последовательность «..=» и двухсимвольная «=>», конфликтующие с базовыми операторами присваивания и диапазона.

Реализация. Логика повторяет вариант 10: порядок проверок «три – два – один». Особенность варианта – откат при неудаче: прочитав «..», лексер не может вернуть токен сразу, пока не убедится, что третий символ не «=». Проще всего отложить решение: проверить три символа через peek() без продвижения и лишь затем продвинуть позицию нужное число раз – так потребность в явном откате исчезает, и в отчёте стоит объяснить, почему.

Обязательные негативные тесты: последовательность «=> = >» – три оператора, а не один; вход 1.. =5 – диапазон и присваивание, отделённые пробелом; вход 1..=5 должен разобраться как «1», «..=», «5».

Вариант 13. Стрелочный оператор -> и доступ к полю через точку

Суть изменения. Добавляется оператор доступа к полю структуры «->», а точка используется и как разделитель вещественных чисел, и как оператор доступа: obj->field, число 3.14.

Реализация. В read_operator() символ «-» сначала проверяется на пару «->» (peek(1) == ">"), и лишь затем возвращается как арифметический оператор вычитания. Точка в ветке чисел уже защищена условием peek(1).isdigit(); вход obj.x после идентификатора выдаёт «.» как одиночный оператор – убедитесь, что SINGLE содержит точку и что трассировка входа a->b корректна.

Обязательные негативные тесты: a-> b – стрелка разорвана пробелом: это оператор «-» и сравнение «>»? Согласуйте правила; вход a->>b – «->» плюс «>»; вход 3.-2 – точка за которой минус: обсуждается, что число «3.» некорректно.

Вариант 14. Идентификаторы с префиксами $ и @

Суть изменения. Специальные идентификаторы начинаются с «$» или «@»: $temp, @counter. Префикс – обязательная часть имени; символ вне идентификатора недопустим.

Реализация. В tokenize() символы «$» и «@» получают собственную ветку, из которой вызывается read_identifier() с флагом префикса. Внутри метода после чтения префикса требуется хотя бы один символ имени; далее сбор идёт по общим правилам. Важно: «@» конфликтует с

вариантом 8 – в своей группе используйте только один из этих вариантов, что стоит отразить в отчёте.

Обязательные негативные тесты: символ «$» в одиночку – ошибка; вход @123 – префикс без имени; вход $$x – второй «$» не является частью имени.

Вариант 15. Словесные операторы «и», «или», «не»

Суть изменения. Логические операторы && || ! заменяются словесными: «и», «или», «не». Лексически это слова, поэтому на первый взгляд они попадают в категорию идентификаторов.

Реализация. Изменяется только read_identifier(): после сбора полной лексемы проверка расширяется – если слово входит в множество WORD_OPS = {"и", "или", "не"}, возвращается токен OP. Ключевая тонкость варианта: проблема границ слова решается сама собой принципом максимального совпадения – read_identifier прочитает все буквы подряд, поэтому «илинет» станет одним идентификатором, а не оператором «или» и «нет». Однако если студент решит распознавать операторы до чтения идентификатора, по первому символу «и», – «илинет» разломается на два токена. Именно эта ловушка – главный материал варианта и обязательный пункт обсуждения на защите.

Обязательные негативные тесты: вход «илинет» – один идентификатор, а не два токена; слово «И» – регистрозависимость превращает его в идентификатор; слово «не» в одиночку – лексически корректно, проверьте, что лексер не пытается оценивать синтаксис (отсутствие операнда – уже не его задача).

6. Требования к отчёту и защите

Отчёт по работе должен содержать следующие разделы:

1. титульный лист; формулировка цели и варианта задания;
2. описание словаря языка: перечень категорий токенов, регулярные выражения для каждой категории;
3. схемы алгоритмов распознавания (не менее двух: для идентификатора и для числового литерала своего варианта);
4. листинги реализации с пояснениями;
5. результаты тестирования: не менее 10 позитивных и 5 негативных примеров; для каждого – вход, полученная таблица токенов или сообщение об ошибке;
6. выводы: что реализовано, какие трудности возникли, сравнение выбранного подхода с альтернативными.

Типовые вопросы на защите:

- Чем лексема отличается от токена? Зачем токену хранить координаты?
- Почему при разборе операторов сначала проверяются двухсимвольные варианты?
- Как ваш лексер обрабатывает конец файла внутри строкового литерала и внутри блочного комментария?
- Попадают ли комментарии в таблицу токенов и почему?

- Какова вычислительная сложность лексического анализа и может ли она деградировать на отдельных входах?
- Что нужно изменить в лексере, чтобы добавить новый тип литералов?
- Как лексер ведёт себя, если вход закончился на середине накопления лексемы?
- Почему для описания лексики достаточно регулярных выражений, а для синтаксиса – нет?

7. Контрольные вопросы и ответы

1. Дайте определения понятий «лексема», «токен», «шаблон», «словарь языка». Приведите примеры для входа x = x1 + 3.14;

Лексема – минимальная значимая последовательность символов исходного текста; токен – её внутреннее представление (тип, значение, позиция); шаблон – правило распознавания лексем категории; словарь – совокупность всех шаблонов и ключевых слов. Во входе x = x1 + 3.14 лексемы: x (ID), = (OP), x1 (ID), + (OP), 3.14 (FLOAT); шаблон ID – [A-Za-z_][A-Za-z0-9_]*; словарь – весь перечень таких правил вместе с ключевыми словами.

2. Изобразите полную цепочку этапов обработки программы. Какие данные передаются между лексером и синтаксическим анализатором?

Цепочка: исходный текст → лексер → поток токенов → синтаксический анализатор → дерево разбора → семантический анализ → генерация кода. Лексер передаёт парсеру последовательность токенов; парсер лексеру обычно ничего не передаёт, но в практических компиляторах работает «по требованию»: запрашивает следующий токен по мере необходимости. Благодаря этому лексеру не нужно держать весь файл в памяти.

3. Что такое регулярное выражение и какую роль оно играет при описании словаря языка?

Регулярное выражение – формальная запись правила «из каких символов и в каком порядке состоит лексема»: перечисления символов и диапазоны ([0-9]), повторения (*, +), необязательность (?) и выбор (|). В лабораторной оно выполняет роль спецификации категории лексем: по нему пишется код распознавания и строятся тестовые примеры. Существенно, что регулярных выражений достаточно именно для лексики; вложенные конструкции (скобки, вложенные блоки) ими не описываются.

4. Сформулируйте словами правило распознавания числового литерала с экспоненциальной частью. Какие входы обязательны в негативных тестах?

Правило: читаем цифры; если встретили точку и за ней цифра – читаем дробную часть; если далее встретили e или E – читаем необязательный знак и хотя бы одну цифру; если цифр после e (или после точки, или после префикса 0x) не оказалось – ошибка. Обязательные негативные тесты: 1e без цифр, 0x без цифр, 1. без дробной части, точка без целой части.

5. Сформулируйте принцип максимального совпадения. Приведите вход, на котором он обязателен, и вход, где он приводит к неожиданному результату.

Если лексему можно завершить и раньше, и позже, выбирается самое длинное совпадение. Обязателен на входе a<=b: иначе он распадётся на < и =. Неожиданный результат даёт вход 1..5 из

варианта 5: по принципу максимума точка примыкает к числу и получается вещественный литерал 1., а затем .5 – поэтому в таких языках правило приходится уточнять отдельно.

6. Что такое просмотр вперёд (lookahead)? В каких местах вашего лексера он необходим?

Это чтение символа, стоящего дальше текущего, без продвижения позиции. Необходим: при различении /, // и /* (нужен один символ вперёд); при разборе двухсимвольных операторов ==, <=, += и других; при проверке, стоит ли за точкой цифра (отличить 3.14 от завершающего 3.); в варианте 15 – при проверке границ словесных операторов.

7. Классифицируйте лексические ошибки. Как формировать сообщение об ошибке, чтобы оно было полезно пользователю?

Ошибки распадаются на группы: незакрытые конструкции (строка, блочный комментарий), некорректные литералы (0x без цифр, экспонента без цифр), недопустимые символы (например, @ в базовом языке), нарушения границ лексем (перевод строки внутри строки). Полезное сообщение содержит позицию (строка, столбец), класс ошибки и короткое описание ожидаемого: «строка 12, столбец 8: ожидаются цифры после 0x». Продолжение разбора после ошибки позволяет найти все ошибки за один прогон.

8. Сравните ручной лексер и лексер на регулярных выражениях: трудоёмкость, производительность, расширяемость.

Ручной лексер требует больше кода, но даёт полный контроль и наглядность – на нём видны все границы и ловушки. Лексер на re короче и ближе к спецификации словаря, но медленнее на больших файлах и плохо выражает контекст (вложенные комментарии, директивы «только в начале строки»). Расширяемость: в ручном лексере новая категория – новая ветка разбора; в re – новое правило, но с риском конфликтов порядка применения шаблонов.

9. Почему проверку ключевых слов удобно выполнять после накопления всей лексемы?

Потому что ключевое слово неотличимо от идентификатора по первому символу: «если» начинается так же, как гипотетический идентификатор «если_2». Лексер сначала собирает полную лексему по общему правилу идентификатора, а затем однократно проверяет её по множеству ключевых слов – это одна операция поиска вместо отдельных проверок на каждом символе.

10. Почему баланс скобок – не задача лексера, а задача синтаксического анализатора?

Баланс скобок – свойство структуры программы, а не отдельной лексемы: и «(», и «)» – корректные токены, каждый сам по себе. Лексер работает только с тем, что видит в пределах одной лексемы, и не помнит, какие скобки встречались раньше. Соответствие скобок проверяет синтаксический анализатор, который видит всю последовательность токенов.

8. Список литературы

1. Ахо, А. Компиляторы: принципы, технологии и инструментарий / А. Ахо, М. Лам, Р. Сети, Дж. Ульман. – 2-е изд. – Москва : Вильямс, 2020. – 1312 с.
2. Лутц, М. Изучаем Python. Т. 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Диалектика, 2019. – 832 с.

3. re – Regular expression operations : документация Python Software Foundation. – URL: https://docs.python.org/3/library/re.html (дата обращения: 07.11.2026).
4. PLY (Python Lex-Yacc) : документация проекта. – URL: https://www.dabeaz.com/ply/ (дата обращения: 07.11.2026).

Список оформлен в соответствии с ГОСТ Р 7.0.100-2018; даты обращения к электронным ресурсам уточняются при сдаче отчёта.